Help & Support System — Backend & DB Architecture Proposal

Strategy Analysis: Standalone Support Service vs. Existing OTT Monolith Extension

Target Audience: Engineering Management, Backend Tech Leads, Architecture Committee

Domain: Support Desk, Realtime Chat, AI Auto-triaging, Ticketing

EXECUTIVE RECOMMENDATION: Hybrid Microservice (Dedicated Backend + Database)
We recommend developing the **Filmytell Help & Support platform as an independent backend service with a dedicated PostgreSQL database**, connected to Filmytell Core via **JWT Auth Federation** and **Internal Context APIs**.
Key Justification: Isolates streaming playback servers from heavy support loads (WebSockets, file uploads, AI queues), enables rapid independent deployments, and provides native tooling for ticket lifecycle state machines.

1. Workload Divergence: OTT Streaming Core vs. Help & Support Domain

The OTT video streaming platform and internal Help & Support service have fundamentally conflicting infrastructure demands:

Engineering Factor	Core OTT Streaming Backend	Help & Support Backend Service
Workload Pattern	High read throughput, low latency catalog browsing, heavy CDN video traffic.	Long-lived WebSockets, live agent chat, periodic bulk reporting, state updates.
Data Domain	Media catalogs, playback streams, watch history, subscriptions, payment invoices.	Ticket lifecycles, delegations, SLAs, chat threads, KB articles, audit logs.
AI / Automation	Recommendation engines & catalog search.	LLM ticket auto-triaging, sentiment detection, auto-reply drafting, vector search.
Failure Blast Radius	Critical: Server degradation immediately interrupts customer video playback.	Operational: High support load must never degrade user streaming experience.

2. Strategic Options Evaluation Matrix

Option A: Extend Existing OTT Backend & DB	Option B: Dedicated Service & DB (Recommended)
Pros: • No new server instances to deploy initially. • Direct foreign key joins to existing user/payment tables. **Risks & Drawbacks:** • **Shared Blast Radius:** Complex support reporting queries or large screenshot uploads could lock tables and degrade streaming APIs. • **Tied Release Cycles:** Any support bug fix requires redeploying the core OTT platform. • **Realtime Limits:** Hard to run thousands of long-lived chat WebSockets on existing REST backend. • **Security Risk:** Support agents operate within the core customer database context.	**Pros:** • **Zero Risk to Streaming:** Complete operational and database isolation. • **Modern Realtime Stack:** Built with WebSockets, Redis pub/sub, and async job queues (BullMQ/Celery) for AI processing. • **High Deployment Velocity:** Independent CI/CD pipeline allows daily updates to support workflows without touching OTT production. • **Strict RBAC:** Internal support agent tokens and roles are isolated. **Tradeoff:** Requires maintaining a separate service & DB instance.

3. Target Architecture & Integration Blueprint

The standalone Help & Support service interfaces cleanly with Filmytell OTT via four core integration pillars:

Integration Pillar	Protocol / Method	Implementation Specification
1. Unified Authentication	Shared JWT / SSO Verification	Support backend validates client JWTs using Filmytell's public key or introspection endpoint. Zero duplicated credentials.
2. Customer Context	Internal Service-to-Service REST	When agents view a ticket, the backend calls GET /ott/api/users/{id} to pull active plan, device type, and recent playback logs.
3. Review & Rating Sync	Periodic Poller / Webhook	Consumes existing /ott/api/RatingAndRewiew/getAll/pages to automatically ingest 1-2 star reviews into actionable support tickets.
4. Media Attachments	S3 / GCS Presigned URLs	User screenshots and video clips upload directly to object storage with time-limited signed URLs, preventing server memory bloat.

4. Recommended Technology Stack

Architecture Layer	Recommended Technology	Engineering Justification
Backend Runtime	**Node.js (NestJS / Fastify)** *Alternative: Python FastAPI*	High I/O concurrency, built-in WebSocket support for live chat, structured modular architecture, seamless integration with AI SDKs.
Primary Database	**PostgreSQL 16+**	ACID compliance for ticket states, JSONB for dynamic device telemetry, and pgvector for semantic Knowledge Base search.
Cache & Job Queue	**Redis + BullMQ**	Handles active agent presence, WebSocket pub/sub clustering, SLA breach countdown timers, and async AI auto-triaging queues.
AI / LLM Engine	**Gemini API / OpenAI**	Automated ticket category classification, sentiment analysis, priority assignment, and suggested smart replies for support agents.

5. Phased Implementation Roadmap

• **Phase 1: Core Service Scaffold & Auth Integration (Weeks 1–2):** Setup repository, configure PostgreSQL schema (Tickets, Messages, KB, Users), implement JWT validation middleware connected to Filmytell Auth, build core ticket CRUD and Kanban endpoints.
• **Phase 2: Real-time Messaging & Review Ingestion (Weeks 3–4):** Deploy WebSocket gateway for live agent-user chat, integrate with /RatingAndRewiew/getAll/pages for automated ticket creation, setup S3/GCS presigned upload endpoints.
• **Phase 3: AI Triaging & SLA Engine (Weeks 5–6):** Deploy BullMQ workers for SLA escalation timers, connect LLM pipeline for ticket categorization and smart suggestions, complete performance and security load tests.

6. Immediate Action Items for Team Alignment

1. **Architecture Sign-Off:** Align engineering management and backend leads on the dedicated service + PostgreSQL approach.
2. **Select Backend Framework:** Finalize team preference between Node.js (NestJS/Fastify) and Python (FastAPI).
3. **Expose Context Endpoint:** Backend team exposes/verifies GET /ott/api/users/{id} for customer profile context retrieval.
4. **Infrastructure Provisioning:** Provision dedicated staging PostgreSQL, Redis, and S3/GCS bucket on cloud infrastructure.